Manual Técnico – Proyecto Oráculo 6 de noviembre del 2026

Manuel Alejandro Portilla López

1. Estructura del Repositorio y Entorno

El repositorio oficial del proyecto está organizado en la siguiente estructura de archivos:

- **proyecto_oraculo.ipynb:** Es el notebook principal. En la Celda 4 se implementó el algoritmo heurístico de búsqueda local por Descenso de Coordenadas.
- **entrega.json:** Archivo JSON exportado que contiene la configuración óptima final descubierta por el algoritmo.
- **oraculo.py y ayudas.py:** Módulos base que contienen la lógica de evaluación.

Requisitos del Entorno:

- **Entorno de Ejecución:** Google Colab con la GPU NVIDIA T4.

2. Guía de Ejecución Secuencial

- **Paso 1 - Carga de Recursos:** Subir los archivos oraculo.py, ayudas.py y datos_visibles.json al colab.
- **Paso 2 - Inicialización del Entorno:** Ejecutar en orden las celdas 1 a 3, reiniciando la celda 1 después de su primera ejecución.
- **Paso 3 - Ejecución del Motor Heurístico:** Ejecutar la Celda 4. El algoritmo de Descenso de Coordenadas iniciará el proceso de exploración optimizada en el espacio de configuraciones.
- **Paso 4 - Validación y Exportación:** Ejecutar las celdas finales del cuaderno para inspeccionar la trazabilidad de fallos, graficar la curva de precisión y exportar el archivo entrega.json.

3. Explicación de la Solución (Celda 4)

Se implementó un algoritmo heurístico de búsqueda local por descenso de coordenadas. En lugar de explorar combinaciones aleatorias, el algoritmo optimiza una ranura o categoría a la vez, conservando el estado óptimo alcanzado en las ranuras anteriores.

Código implementado en la celda 4:

```
import copy
from ayudas import curva

INSTANCIAS = busqueda

# 1. Extracción dinámica del espacio válido
categorias = list(CONFIGS[0].keys())
opciones_validas = {cat: list(set(c[cat] for c in CONFIGS)) for cat in
categorias}

# 2. Evaluación base (Punto de partida)
c_actual = CONFIGS[0].copy()
r_actual = oraculo.evaluar(c_actual, INSTANCIAS, semilla=1)

mejor = (r_actual.precision, c_actual.copy())
historial = [r_actual.precision]

print(f"Inicio de búsqueda (Descenso de Coordenadas) - Precisión base:
{mejor[0]:5.1%}")

# 3. Motor Heurístico: Una sola pasada
for cat in categorias:
```

```
        print(f"\n--- Optimizando ranura: {cat} ---")
        for valor in opciones_validas[cat]:
            if valor == c_actual[cat]:
                continue

            c_candidata = c_actual.copy()
            c_candidata[cat] = valor

            # Consulta al modelo
            r_cand = oraculo.evaluar(c_candidata, INSTANCIAS, semilla=1)

            # Criterio Greedy: Solo aceptamos mejoras estrictas
            if r_cand.precision > mejor[0]:
                mejor = (r_cand.precision, c_candidata.copy())
                c_actual = c_candidata.copy()
                print(f" [!] Mejora en {cat}={valor} -> Precisión:
{mejor[0]:5.1%}")

            # 4. Registro estable para curva de convergencia ascendente
            historial.append(mejor[0])

    print("\nMejor configuración:", mejor[1])
    curva(historial)
```

4. Análisis del Código

- **Paso 1 - Obtención de opciones válidas:** Para evitar errores durante la evaluación en la GPU (como pasarle un tipo de dato o valor no permitido a la temperatura), el código recorre la lista CONFIGS original y extrae automáticamente solo los valores permitidos para cada categoría dentro del diccionario opciones_validas.
- **Paso 2 - Línea base:** Se evalúa la primera configuración disponible (CONFIGS[0]) para obtener una precisión inicial. Este valor queda guardado en mejor[0] y sirve como el punto de comparación pivote para todas las pruebas siguientes.
- **Paso 3 - Búsqueda por Descenso de Coordenadas:** El ciclo optimiza el prompt ranura por ranura (rol, estrategia, formato, estilo, cierre y temperatura). En cada ranura, el algoritmo deja quietas las demás categorías y prueba únicamente los valores alternativos de la ranura actual. Si alguna opción logra superar la precisión base (r_cand.precision > mejor[0]), la convierte de inmediato en la nueva configuración de referencia (c_actual).
- **Paso 4 - Historial para la gráfica de progreso:** En cada intento se guarda mejor[0] en la lista historial. Al registrar siempre la mejor precisión alcanzada hasta el momento se logra que la función curva() dibuje una gráfica en forma de escalera ascendente que muestra con cómo fue mejorando el algoritmo paso a paso.

5. Formatos de Entrada/Salida y Limitaciones

- **Formato de Entrada:** El sistema recibe la variable CONFIGS (una lista de diccionarios con las opciones de prompt) y el archivo datos_visibles.json (que contiene las instancias de prueba con sus respectivas reglas a cumplir).
- **Formato de Salida:** El algoritmo arroja por consola el registro de mejoras iterativas y exporta un archivo entrega.json que contiene un único diccionario con los índices de la configuración ganadora y la semana.

- **Limitaciones del Algoritmo:** Al ser una Búsqueda Local de una sola pasada (criterio Greedy), el algoritmo se supone rápido y protege la memoria de la GPU, pero tiene la limitación de que puede quedarse estancado en un óptimo local. Si dos categorías del prompt dependen fuertemente entre sí para funcionar bien, el algoritmo podría no descubrir esa sinergia porque solo cambia una variable a la vez.